

Task 1.1: Superkey and Candidate Key Analysis

Relation A: Employee:

Schema: Employee (EMPID, SSN, Email, Phone, Name, Department, Salary)

1. Six Different Superkeys:

{EMPID}, {SSN}, {Email}, {EMPID, Name}, {SSN, Email}, {EMPID, SSN, Email, Phone, Name, Department, Salary}

2. Candidate keys:

{EMPID}, {SSN}, {Email}, {Phone}

3. Primary Key Selection and Justification:

Selection: {EMPID}

Justification: I would choose EMPID as the primary key because it is unique for each employee and can identify each employee easily. It is also simple, stable, and does not contain sensitive personal information such as SSN. In addition, EMPID is more convenient to use than Email or Phone because these values can change over time.

4. Can two employees have the same phone number?

Answer: No, based on the given data, two employees cannot have the same phone number because each employee in the table has a unique phone number. This means that the phone number can distinguish one employee from another in the given dataset. However, in a real database, phone numbers should only be considered unique if a UNIQUE constraint is defined for the Phone attribute.

Relation B: Course Registration:

Schema: Registration (StudentID, CourseCode, Section, Semester, Year, Grade, Credits)

1. Minimum Attributes Needed for Primary Key:

{StudentID, CourseCode, Semester, Year}

2. Necessity of Each Attribute:

StudentID – identifies the specific student who registered for the course.

CourseCode – identifies the specific course that the student is taking.

Section – identifies the particular section of the course.

Semester – shows the semester in which the student registered for the course.

Year – identifies the academic year of the registration.

3. Additional Candidate Keys:

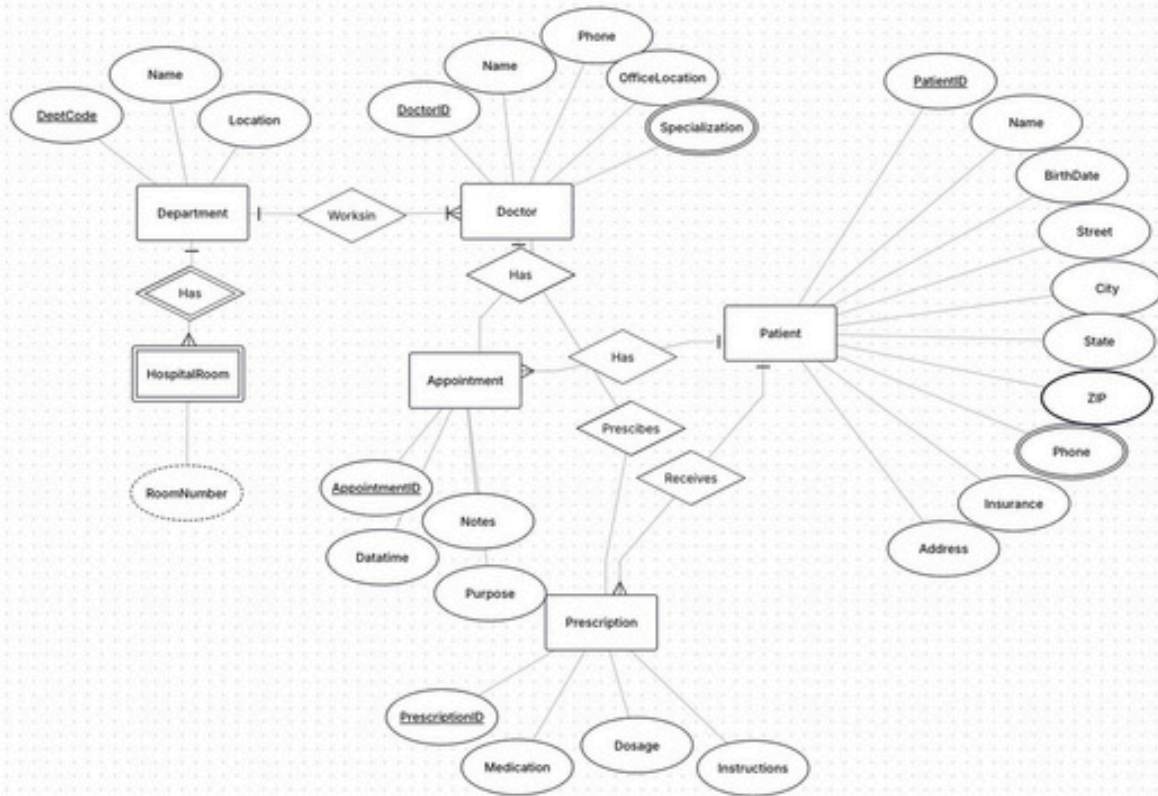
There are no additional candidate keys because the combination of StudentID, CourseCode, Section, Semester, and Year already uniquely identifies each registration. This combination is the only candidate key for the given relation.

Task 1.2: Foreign Key Design

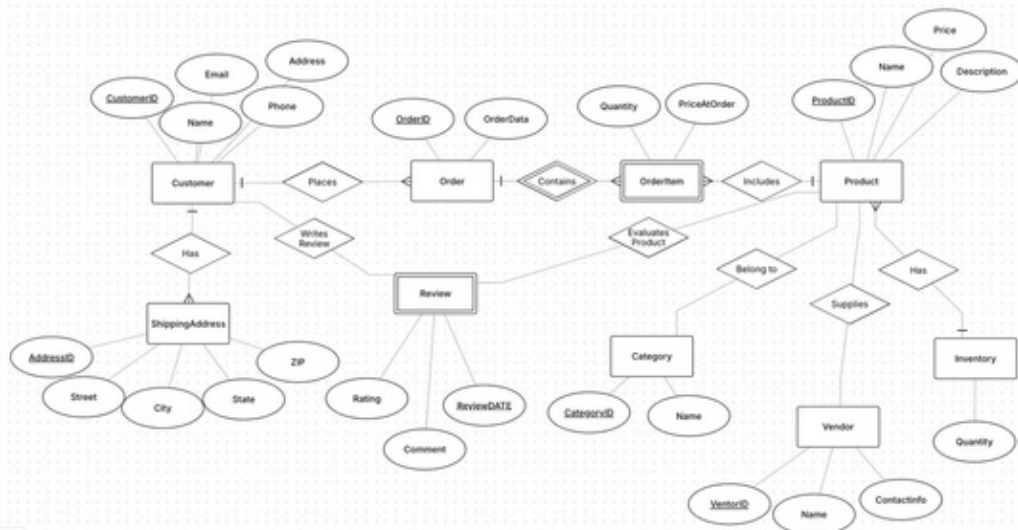
Foreign Key Relationships:

Child Table	Foreign Key Attribute	Parent Table	Referenced Primary Key	Description
Student	AdvisorID	Professor	ProID	Identifies the professor who is the student's advisor.
Professor	Department	Department	DeptCode	Identifies the department where the professor works.
Course	DepartmentCode	Department	DeptCode	Identifies the department that offers the course.
Department	ChairID	Professor	ProID	Identifies the professor who is the department chair.
Enrollment	StudentID	Student	StudentID	Identifies the student enrolled in the course.
Enrollment	CourseID	Course	CourseID	Identifies the course in which the student is enrolled.

Task 2.1: Hospital Management System



Task 2.2: E - commerce Platform



Task 4.1: Denormalized Table Analysis

1.

1. Functional Dependencies (FDs):

2. FD1: StudentID -> StudentName, StudentMajor
3. FD2: ProjectID -> ProjectTitle, ProjectType, SupervisorID, StartData, EndData
4. FD3: SupervisorID -> SupervisorName, SupervisorDept
5. FD4: StudentID, ProjectID -> Role, HoursWorked

2. Redundancies and Anomalies:

Redundancy: Student details (StudentName, StudentMajor), project details (ProjectTitle, ProjectType), and supervisor details are duplicated for every project assignment record.

Insert Anomaly: Cannot insert a new student into the database if they are not currently assigned to a project because (StudentID, ProjectID) forms the composite primary key (Primary Key attributes cannot be NULL).

Delete Anomaly: Deleting a student's project record might accidentally remove the only record containing information about that specific project or supervisor.

Update Anomaly: Updating a supervisor's department requires updating multiple rows across the table, risking data inconsistency.

3. 1NF Compliance:

Assuming all fields hold atomic values, the relation is already in 1NF.

4. 2NF Decomposition:

Primary Key : {StudentID, ProjectID}

Partial Dependencies FD1 AND FD2 depend on parts of the composite key.

2NF Relations:

1. Student (StudentID, StudentName, StudentMajor) – PK: StudentID
2. Project (ProjectID, ProjectTitle, ProjectType, SupervisorID, SupervisorName, SupervisorDept, StartDate, EndDate) – PK: ProjectID
3. StudentProjectAssignment (StudentID, ProjectID, Role, HoursWorked) – PK: (StudentID, ProjectID)

5. 3NF Decomposition:

Transitive Dependency: In Project, ProjectID → SupervisorID → SupervisorName, SupervisorDept

Final 3NF Schemas:

Student (StudentID, StudentName, StudentMajor)

Supervisor (SupervisorID, SupervisorName, SupervisorDept)

Project (ProjectID, ProjectTitle, ProjectType, SupervisorID, StartDate, EndDate)

FK: SupervisorID → Supervisor (SupervisorID)

StudentProjectAssignment (StudentID, ProjectID, Role, HoursWorked)

FK1: StudentID → Student(StudentID)

FK2: ProjectID → Project(ProjectID)

Task 4.2: Advanced Normalization

1. Primary Key :

Primary Key: {StudentID, TimeSlot}

Explanation: A student can attend only one course section in a given time slot, uniquely identifying the tuple.

2. List of All Functional Dependencies:

FD1: StudentID → StudentMajor

FD2: CourseID → CourseName

FD3: InstructorID -> InstructorName

FD4: Room -> Building

FD5: InstructorID, TimeSlot -> CourseID, Room

FD6: Room, TimeSlot -> InstructorID, CourseID

3. BCNF Verification:

The table is not in BCNF because functional dependencies exist where the determinant is not a superkey (e.g., StudentID -> StudentMajor)

4.

1. BCNF Decomposition;
2. Student (StudentID, StudentMajor)
3. Course (CourseID, CourseName)
4. Instructor (InstructorID, InstructorName)
5. RoomLocation (Room, Building)
6. CourseOffering (CourseID, TimeSlot, InstructorID, Room)
7. Enrollment (StudentID, TimeSlot, CourseID)

Task 5.1: Real-World Application

